

Informe del módulo de posicionamiento

CSDI RAG Engine

1 Módulo de posicionamiento

El módulo de posicionamiento es el componente encargado de determinar el orden final en que la información recuperada debe ser entregada al usuario y al módulo RAG. En el sistema, el posicionamiento no se limita a mostrar los documentos en el mismo orden en que son encontrados por un único recuperador. La arquitectura combina señales léxicas, semánticas, de procedencia, de frescura y de método de recuperación para construir una prioridad final de presentación.

La función de este módulo debe distinguirse de la recuperación. El recuperador obtiene candidatos potencialmente relevantes a partir de la consulta del usuario; el posicionador decide cómo ordenar esos candidatos y qué señales adicionales deben influir en su ubicación final. Esta separación es importante porque en un sistema híbrido un resultado puede ser recuperado por distintas vías: coincidencia exacta de términos, similitud semántica, búsqueda en caché web o una combinación de varias de ellas. El módulo de posicionamiento permite convertir esas señales heterogéneas en una lista final ordenada, explicable y adecuada para ser presentada en una interfaz visual.

Rol dentro de la arquitectura general. El flujo de posicionamiento comienza después de recibir una consulta en los endpoints de búsqueda o de RAG. Primero se ejecutan los recuperadores de primera etapa: BM25 para recuperación léxica y el recuperador vectorial para similitud semántica. Luego, sus listas de resultados se fusionan mediante una estrategia híbrida. Posteriormente, el módulo de posicionamiento toma los candidatos ya recuperados, calcula señales adicionales para cada fragmento y produce una prioridad final de visualización.

En la implementación, este flujo afecta principalmente tres puntos de entrada: `POST /api/v1/search`, `POST /api/v1/search/web-cache` y `POST /api/v1/rag/query`. En los dos primeros casos se devuelve una lista de resultados ordenada por prioridad de presentación. En el caso de RAG, las fuentes utilizadas para generar la respuesta también reciben señales de posicionamiento, de modo que la explicación de la respuesta pueda mostrar primero las fuentes más importantes.

Ranking léxico mediante BM25. La primera señal de posicionamiento proviene del recuperador BM25. Este modelo trabaja sobre el índice invertido construido por el módulo de indexación. Para cada consulta, los términos se normalizan y se buscan en las listas de postings correspondientes. Cada documento candidato recibe una puntuación de acuerdo con la frecuencia del término en el documento, la longitud del documento, la longitud promedio del corpus y la frecuencia documental del término.

BM25 es especialmente útil en el dominio de documentación técnica porque muchos usuarios formulan consultas con nombres de funciones, clases, módulos, errores o palabras reservadas. En esos casos, la coincidencia léxica exacta es una señal fuerte de relevancia. Por ejemplo, una consulta como `python decorators`, `asyncio task` o `array map` contiene términos que suelen aparecer literalmente en los documentos que responden la necesidad de información.

La puntuación BM25 utilizada por el sistema responde a la forma general:

$$\text{score}_{BM25}(D, Q) = \sum_{q_i \in Q} \text{IDF}(q_i) \cdot \frac{tf(q_i, D)(k_1 + 1)}{tf(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}}\right)}$$

Esta señal aporta precisión en términos exactos y sirve como una de las bases del ranking híbrido. Sin embargo, BM25 por sí solo no captura bien relaciones semánticas cuando la consulta y el documento usan vocabularios distintos. Por esa razón se combina con recuperación vectorial.

Ranking semántico mediante recuperación vectorial. La segunda señal principal proviene del recuperador vectorial. En este caso, los fragmentos de documentos se representan mediante embeddings densos generados con un modelo de *Sentence Transformers*. La consulta del usuario también se transforma en un vector y se compara contra los vectores almacenados en el índice FAISS.

El sistema usa embeddings normalizados y búsqueda por producto interno, lo cual aproxima la similitud coseno entre consulta y documentos. Esta señal permite recuperar fragmentos semánticamente cercanos aunque no compartan todos los términos exactos de la consulta. Por ejemplo, una consulta sobre “modificar el comportamiento de una función” puede recuperar documentos sobre decoradores aunque la palabra **decorator** no aparezca explícitamente en la consulta.

La recuperación vectorial mejora la cobertura semántica del sistema, pero puede perder precisión en términos técnicos raros, nombres exactos de APIs o errores específicos. Por tanto, el diseño no elige entre BM25 y vectores, sino que los combina.

Ranking híbrido mediante Reciprocal Rank Fusion. El ranking híbrido se implementa con *Reciprocal Rank Fusion* (RRF). Esta técnica fusiona varias listas ordenadas tomando en cuenta la posición que ocupa cada documento en cada lista, no la escala absoluta de sus puntuaciones. Esto es útil porque las puntuaciones de BM25 y las puntuaciones vectoriales no son directamente comparables: BM25 produce valores derivados de frecuencias e IDF, mientras que el recuperador vectorial produce similitudes en un espacio denso.

Para cada documento d , la puntuación RRF se calcula como:

$$\text{RRF}(d) = \sum_{r \in R} w_r \cdot \frac{1}{k + \text{rank}_r(d)}$$

donde R es el conjunto de rankings a fusionar, w_r es el peso asignado a cada ranking, k es una constante de suavizado y $\text{rank}_r(d)$ es la posición del documento dentro del ranking r . En el sistema se usa una fusión ponderada con pesos por defecto de 0.3 para BM25 y 0.7 para recuperación vectorial.

Esta elección favorece la señal semántica, que resulta importante para consultas en lenguaje natural, pero conserva la señal léxica para términos técnicos exactos. Un documento que aparece bien posicionado tanto en BM25 como en búsqueda vectorial tiende a obtener mejor puntuación final que un documento recuperado por una sola vía. Esto hace que el ranking híbrido sea más robusto que cualquiera de los recuperadores individuales.

Identificación del método de recuperación. Además de ordenar los documentos, el sistema conserva la información sobre cómo fue encontrado cada resultado. Cada candidato puede marcarse como recuperado por **bm25**, por **vector**, por **hybrid** cuando aparece en ambas listas, o por **web-cache** cuando proviene de la caché web. Esta señal se expone como **retrieval_method**.

La utilidad de esta información es doble. Primero, permite explicar al usuario o al evaluador por qué un resultado fue considerado relevante. Segundo, permite incorporar el método de recuperación

como una señal adicional en el posicionamiento final. En la fórmula de prioridad visual, los resultados híbridos reciben una bonificación porque fueron apoyados por más de una estrategia de recuperación.

Re-ranking de segunda etapa. El sistema también contempla un re-ranker opcional basado en un *cross-encoder*. A diferencia del recuperador vectorial, que codifica consulta y documento por separado, el cross-encoder evalúa pares (*consulta, documento*) de forma conjunta. Esto permite calcular una puntuación de relevancia más precisa para un conjunto reducido de candidatos.

El uso previsto es el siguiente: el ranking híbrido recupera los primeros N candidatos y el re-ranker vuelve a ordenarlos para seleccionar los fragmentos que pasarán al prompt de RAG. Esta etapa resulta especialmente útil cuando se necesita alta precisión en la evidencia usada para generar respuestas. En términos de posicionamiento, el re-ranking actúa como una segunda etapa más costosa pero más precisa, aplicada únicamente sobre los candidatos ya filtrados por el recuperador híbrido.

Posicionamiento por relevancia normalizada. Una vez obtenida la lista de candidatos, el módulo de posicionamiento calcula una señal **relevance_score**. Esta señal se deriva de la puntuación del ranking híbrido, pero se normaliza al intervalo $[0, 1]$ dentro del conjunto de resultados de la consulta. Si $s(d)$ es la puntuación original de un documento, entonces:

$$\text{relevance_score}(d) = \frac{s(d) - s_{\min}}{s_{\max} - s_{\min}}$$

cuando $s_{\max} > s_{\min}$. Si todos los resultados tienen la misma puntuación, se asigna valor máximo a los documentos con puntuación positiva. La normalización facilita combinar la relevancia con otras señales, como frescura y procedencia, sin depender de la escala interna de RRF.

Esta es la señal de mayor peso en el posicionamiento final. En la implementación representa el 70% de la prioridad de visualización. La razón es que el sistema debe seguir privilegiando la relevancia informacional: un documento reciente o de una fuente confiable no debe superar a otro mucho más pertinente si la consulta realmente apunta al segundo.

Posicionamiento por frescura. La segunda señal incorporada es la frescura del documento. En dominios como tecnología y software, la fecha puede ser relevante porque APIs, versiones, recomendaciones y herramientas cambian con el tiempo. Por esa razón, el sistema intenta obtener fechas editoriales durante el scraping y las propaga hasta los chunks indexados.

El orden de preferencia para calcular la frescura es:

`document_updated_at > published_at > created_at`

La fecha de actualización se prefiere sobre la fecha de publicación porque un documento publicado hace años puede seguir siendo vigente si fue actualizado recientemente. Si no existe fecha editorial, se utiliza la fecha de creación del chunk en el sistema como respaldo.

La extracción de fechas se realiza desde varias fuentes del HTML: metadatos como `article:published_time`, `article:modified_time` y `og:updated_time`; campos `datePublished` y `dateModified` en JSON-LD; etiquetas `time` con atributo `datetime`; y, como último respaldo, la cabecera HTTP `Last-Modified`. Estas fechas se persisten en los documentos fuente, chunks y caché web.

El valor de **freshness_score** se calcula mediante tramos:

$$\text{freshness}(d) = \begin{cases} 1.0, & \text{si la edad es de 0 a 7 días} \\ 0.8, & \text{si la edad es de 8 a 30 días} \\ 0.6, & \text{si la edad es de 31 a 90 días} \\ 0.4, & \text{si la edad es de 91 a 180 días} \\ 0.2, & \text{si la edad es de 181 a 365 días} \\ 0.1, & \text{si la edad es mayor que 365 días} \end{cases}$$

Cuando no existe ninguna fecha disponible, se asigna un valor neutral de 0.5. Esta decisión evita penalizar excesivamente documentos que pueden ser relevantes pero no publican metadatos temporales de forma explícita.

Posicionamiento por procedencia. El sistema distingue entre resultados provenientes del corpus controlado y resultados provenientes de la caché web. Esta señal se expone mediante el campo `source_type`. Actualmente se diferencian principalmente dos categorías:

- **corpus:** documentos adquiridos desde fuentes configuradas y controladas.
- **web_cache:** documentos obtenidos mediante búsqueda web y almacenados para uso posterior.

La procedencia influye en la prioridad de presentación mediante un `source_score`. Los documentos del corpus reciben un valor mayor porque provienen de fuentes definidas explícitamente para el dominio del sistema. Los documentos de caché web son útiles para ampliar la cobertura cuando la información local es insuficiente, pero pueden ser más heterogéneos en calidad, estructura y estabilidad.

Posicionamiento por método de recuperación. Además de la procedencia, el sistema considera el método por el cual fue recuperado cada documento. El campo `retrieval_method` puede tomar valores como `hybrid`, `bm25`, `vector` o `web_cache`. Esta señal se transforma en un `method_score`.

La prioridad más alta se asigna a `hybrid`, porque indica que el documento fue respaldado tanto por la señal léxica como por la semántica. Los documentos recuperados solo por vectores o solo por BM25 también pueden ser relevantes, pero la coincidencia de ambas señales aumenta la confianza del sistema. Los documentos de caché web reciben un tratamiento separado porque su papel principal es complementar el corpus cuando la información local no alcanza.

Cálculo de la prioridad final de visualización. La prioridad final se expone mediante el campo `display_priority`. Esta puntuación combina las señales anteriores mediante una fórmula ponderada:

$$\text{display_priority}(d) = 0.70 \cdot \text{relevance_score}(d) + 0.10 \cdot \text{freshness_score}(d) + 0.10 \cdot \text{source_score}(d) + 0.10 \cdot \text{method_score}(d)$$

El peso mayor se asigna a la relevancia porque la tarea principal del sistema sigue siendo responder la consulta del usuario. La frescura, la procedencia y el método de recuperación actúan como señales auxiliares. Su función es desempatar o ajustar el orden cuando varios documentos tienen relevancia similar, no reemplazar la pertinencia de contenido.

Esta fórmula es deliberadamente explicable. Cada componente puede justificarse durante la evaluación del proyecto y puede modificarse si se observa que un dominio específico requiere más

énfasis en frescura, autoridad de fuente u otra señal. Además, los pesos están separados de la lógica de recuperación, lo que facilita ajustar el comportamiento del posicionamiento sin reescribir los recuperadores.

Ordenamiento final y campo `rank`. Después de calcular `display_priority`, los resultados se ordenan de forma descendente por esta puntuación. El primer resultado de la lista es el de mayor prioridad de presentación. A continuación, el sistema asigna un campo **`rank`**, comenzando en 1, que representa la posición final del resultado.

El frontend no necesita reordenar los resultados. Puede renderizar la lista en el orden recibido desde la API y usar **`rank`** para numeración visual. Esta decisión evita duplicar lógica de posicionamiento en la interfaz y mantiene el criterio de ordenamiento centralizado en el backend.

Campos expuestos por la API. Los endpoints de búsqueda devuelven, además de los datos propios del chunk, un conjunto de campos específicos de posicionamiento:

- **`score`**: puntuación original producida por el ranking híbrido.
- **`relevance_score`**: puntuación de relevancia normalizada en `[0, 1]`.
- **`freshness_score`**: puntuación de frescura calculada a partir de fechas editoriales o de creación.
- **`display_priority`**: prioridad final usada para ordenar la presentación.
- **`rank`**: posición final del resultado en la lista.
- **`source_type`**: procedencia del documento, por ejemplo `corpus` o `web_cache`.
- **`retrieval_method`**: método por el cual fue recuperado, por ejemplo `hybrid`, `bm25`, `vector` o `web_cache`.

Un resultado de búsqueda puede representarse de forma simplificada como:

```
{
  "chunk_id": "python_docs:abc123:0",
  "score": 0.01639,
  "relevance_score": 1.0,
  "freshness_score": 0.8,
  "display_priority": 0.98,
  "rank": 1,
  "source_type": "corpus",
  "retrieval_method": "hybrid",
  "source_id": "python_docs",
  "url": "https://docs.python.org/3/...",
  "title": "Decorators",
  "breadcrumb": "Python Docs > Glossary",
  "text": "A decorator is a function..."
}
```

Integración con el módulo RAG. El módulo RAG también utiliza el posicionamiento. Después de recuperar los fragmentos que se emplearán como contexto, las fuentes asociadas a la respuesta reciben los mismos campos: `relevance_score`, `freshness_score`, `display_priority`, `rank`, `source_type` y `retrieval_method`. De esta forma, la respuesta generada por el LLM puede ir acompañada de una lista de fuentes ordenada por prioridad.

Esta integración es relevante porque en RAG no basta con generar una respuesta: también es necesario mostrar de dónde proviene la evidencia. Si varias fuentes participaron en el contexto, el posicionamiento permite mostrar primero aquellas que el sistema considera más fuertes, ya sea por relevancia, frescura, procedencia o método de recuperación.

Manejo de resultados de caché web. Cuando el detector de insuficiencia determina que el corpus local no contiene información suficiente, el sistema puede consultar la caché web o ejecutar una búsqueda externa. Los documentos obtenidos de esa vía se segmentan, se indexan y pueden participar en búsquedas posteriores. El módulo de posicionamiento los identifica como `web_cache`.

Esta decisión permite ampliar la cobertura del sistema sin mezclar de forma opaca los documentos externos con el corpus principal. El usuario o la interfaz pueden distinguir el origen del resultado, y el backend puede asignar una prioridad ligeramente diferente a los resultados web para reflejar su naturaleza complementaria.

Ventajas del diseño implementado. El diseño del módulo tiene varias ventajas. Primero, combina señales léxicas y semánticas, lo que mejora la robustez del ranking frente a consultas exactas y consultas en lenguaje natural. Segundo, mantiene una explicación clara de cada resultado mediante los campos `source_type` y `retrieval_method`. Tercero, incorpora frescura sin permitir que la fecha domine por completo la relevancia. Cuarto, centraliza el orden final en el backend, por lo que la interfaz solo necesita presentar los resultados en el orden recibido.

Además, el uso de una fórmula ponderada hace que el comportamiento sea ajustable. Si en el futuro el dominio requiere priorizar documentos más recientes, se puede aumentar el peso de `freshness_score`. Si se desea favorecer más el corpus local, se puede modificar `source_score`. Si se incorpora retroalimentación de usuarios, esta puede añadirse como una nueva señal dentro de `display_priority`.

Limitaciones y mejoras futuras. La principal limitación del posicionamiento actual es que algunas señales dependen de la calidad de los metadatos disponibles. No todas las páginas publican fechas de actualización o publicación de forma estándar. En esos casos, el sistema recurre a la fecha de creación local del chunk, que indica cuándo el documento fue incorporado al sistema, pero no necesariamente cuándo fue publicado o actualizado por la fuente original.

Otra limitación es que los pesos de la fórmula son heurísticos. Aunque son razonables y explicables, podrían evaluarse empíricamente con métricas como NDCG, MRR, Precision o Recall usando un conjunto de consultas y juicios de relevancia. También podrían incorporarse señales de popularidad, autoridad de fuente, número de clics, retroalimentación explícita de usuarios o preferencias por dominio.

Como mejora futura, el módulo podría ajustar los pesos dinámicamente según el tipo de consulta. Por ejemplo, consultas sobre versiones recientes de software podrían dar más peso a la frescura, mientras que consultas sobre conceptos estables podrían depender casi por completo de la relevancia textual y semántica.

Conclusión. El módulo de posicionamiento integra el ranking híbrido BM25-vectorial, la fusión mediante RRF, el re-ranking opcional y una etapa final de priorización basada en relevancia, frescura, procedencia y método de recuperación. Con esto, el sistema no solo recupera documentos relevantes, sino que los organiza de manera explícita y defendible para su presentación al usuario y para el soporte de respuestas RAG.